CS405 Project 1: Transformations, Projection & Curves from Paintings

Taha Mehdi (ID 33731) · CS405 Introduction to Computer Graphics, Fall 2026 · Code: github.com/tahamehdidev/cs405-project1 (private), tag project1-final

Painting and application

Painting: II. Selim Türbesi Kapısı, Şevket Dağ (Sakıp Sabancı Museum). I chose it because one picture offers everything the project needs: a doorway whose wall and floor lines recede (clear perspective for the camera work), the smooth orange border of the tiled arch (a clean contour to trace with Bézier curves), and a turned marble column whose profile can be traced and swept into a mesh (bonus). In the scene the painting hangs as a textured quad; a hanging lamp, three blocks and a controlled cube stand in front of it, a curve is traced over the arch, and a 3D copy of the column is pulled out of the wall.

Application (Part E): one WebGPU/WGSL application in plain JavaScript served by Vite (`npm install && npm run dev`). Controls switch the view (main or fixed inspection camera), reset the scene and toggle the debug drawings (world axes, node axes, frustum, control polygons, de Casteljau, Catmull–Rom). A readout shows frame time, CPU time, draw calls and vertices submitted per frame. Code is split into `math/`, `scene/`, `render/` and `ui/`; the README lists every control. All 84 unit tests pass. Conventions: column vectors, p' = M·p, multiply(A, B) = A·B (the right-hand matrix acts first), column-major storage, right-handed world, camera looks down −z, WebGPU depth z in [0, 1].

Part A: T·R·S versus S·R·T

Example (the numbers typed into the application): t = (0.4, 0.2, 0.3), rotation 90° about z, uniform scale 0.3, test point p = (1, 0, 0). Multiplying from the right, T·R·S = T·(R·S) and S·R·T = S·(R·T), with R·t = (−0.2, 0.4, 0.3):

$$T\cdot R\cdot S = \begin{bmatrix} 0 & -0.3 & 0 & 0.4 \\ 0.3 & 0 & 0 & 0.2 \\ 0 & 0 & 0.3 & 0.3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad S\cdot R\cdot T = \begin{bmatrix} 0 & -0.3 & 0 & -0.06 \\ 0.3 & 0 & 0 & 0.12 \\ 0 & 0 & 0.3 & 0.09 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The point p goes to **(0.4, 0.5, 0.3)** under T·R·S (S, then R, then T: (0.3, 0, 0) → (0, 0.3, 0) → (0.4, 0.5, 0.3)) and to **(−0.06, 0.42, 0.09)** under S·R·T (T, then R, then S: (1.4, 0.2, 0.3) → (−0.2, 1.4, 0.3) → (−0.06, 0.42, 0.09)). **Why:** both products have the same 3×3 block, 0.3·R, so with a uniform scale the order does not change rotation or size. They differ in the translation column: in T·R·S it is t itself, (0.4, 0.2, 0.3), because the object is scaled and rotated about its own origin and only then moved. In S·R·T the translation is applied first and is rotated and scaled with the object, so the column becomes S·R·t = (−0.06, 0.12, 0.09) and the object ends up near the origin. Multiplication is not commutative. **The whole project uses M = T·R·S**, which keeps size and rotation about the object's own centre.





Application with the same numbers. Left: M = T·R·S (translation column 0.40, 0.20, 0.30). Right: M = S·R·T (translation column −0.06, 0.12, 0.09). Both matrices are shown in the panel and equal the hand results above.

Part B: scene graph and a hand-verified world matrix

```
gate            level 1  fixed bracket above the doorway
 └ arm           level 2  swings about z
    └ chain      level 3  hinged at the end of the arm
       └ lamp      level 4  spins about y
          └ bead      level 5  orbits the lamp  (the leaf)
```

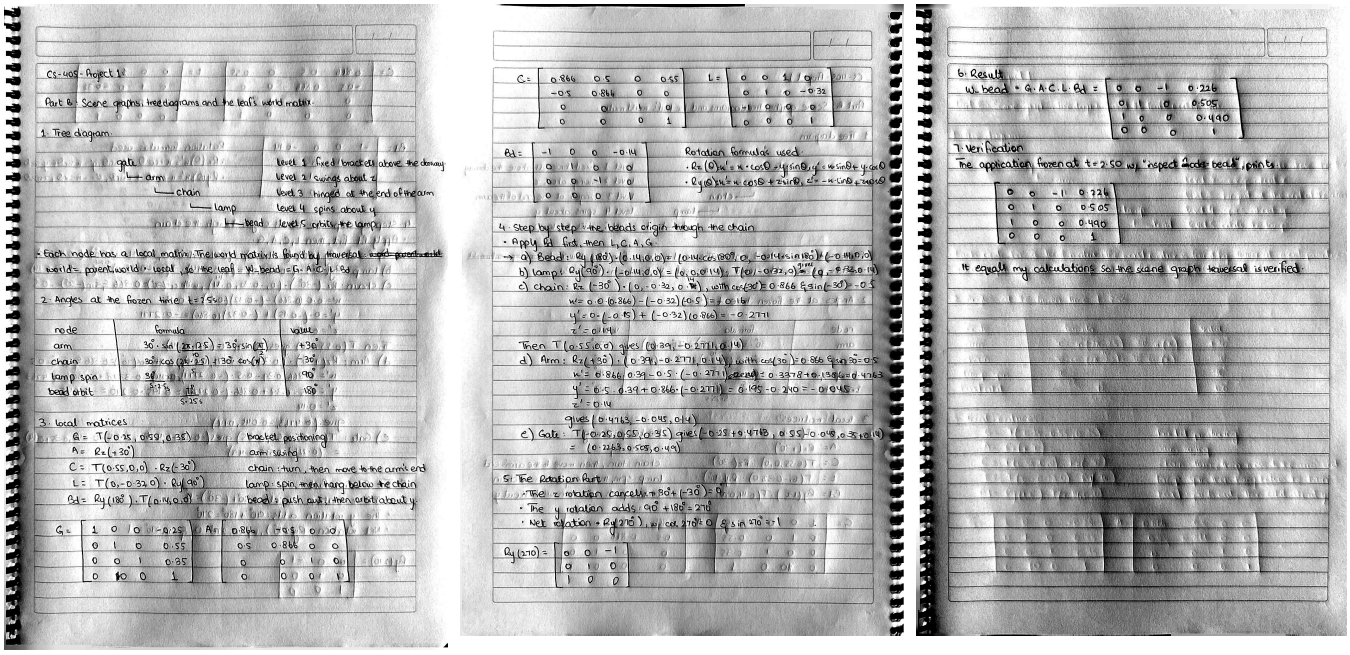
Each node has a local matrix and the world matrix comes from traversal, **world = parent.world · local**, so **W_bead = G·A·C·L·Bd**. At the frozen time t = 2.5 s the angles are arm +30° (30°·sin(π/2)), chain −30° (30°·cos π), lamp 90° (36°/s), bead 180° (72°/s). Local matrices: G = T(−0.25, 0.55, 0.35), A = R_z(30°), C = T(0.55, 0, 0)·R_z(−30°), L = T(0, −0.32, 0)·R_y(90°), Bd = R_y(180°)·T(0.14, 0, 0). Applying them to the bead's origin from the right:

after	computation	point
bead	R_y(180°)·(0.14, 0, 0) = (0.14·cos180°, 0, −0.14·sin180°)	(−0.14, 0, 0)
lamp	R_y(90°)·(−0.14, 0, 0) = (0, 0, 0.14), then T(0, −0.32, 0)	(0, −0.32, 0.14)
chain	R_z(−30°): x = −(−0.32)(−0.5) = −0.16, y = 0.866·(−0.32) = −0.2771; then T(0.55, 0, 0)	(0.39, −0.2771, 0.14)
arm	R_z(+30°): x = 0.866·0.39 + 0.5·0.2771 = 0.4763, y = 0.5·0.39 − 0.866·0.2771 = −0.045	(0.4763, −0.045, 0.14)
gate	T(−0.25, 0.55, 0.35)	**(0.2263, 0.505, 0.49)**

The z rotations cancel (+30° − 30° = 0°) and the y rotations add (90° + 180° = 270°), so the net rotation is R_y(270°) and the world matrix of the bead is

$$W_{bead} = \begin{bmatrix} 0 & 0 & -1 & 0.226 \\ 0 & 1 & 0 & 0.505 \\ 1 & 0 & 0 & 0.49 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The application, frozen at t = 2.50 s with "inspect node = bead", prints the same world matrix (translation 0.226, 0.505, 0.490), so the traversal is verified for a leaf at depth 5. My handwritten working follows (zoom in to read; pages 1 to 3 in order).



Handwritten working for Part B (zoom in to read): tree, angles and local matrices (left); the step-by-step calculation and the rotation part (middle); the result and its verification against the application (right).



The scene at t = 2.5 s with each node's local axes drawn (debug mode); the panel shows the bead's local and world matrix.

Part C: camera and projection

View matrix from lookAt

Given eye e, target t and approximate up u. Camera axes in world coordinates: forward f = (t − e)/|t − e|; right s = (f × u)/|f × u| (first cross product); true up v = s × f (second cross product, already unit length); back b = −f, because the camera looks down −z. A world point p has camera coordinates x_c = s·(p − e), y_c = v·(p − e), z_c = b·(p − e), so with rows s, v, b:

$$V = \begin{bmatrix} s_x & s_y & s_z & -s\cdot e \\ v_x & v_y & v_z & -v\cdot e \\ b_x & b_y & b_z & -b\cdot e \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

So V = R·T(−e). R has orthonormal right-handed rows, hence $R^{-1} = R^{T}$ and $V^{-1} = [R^{T} \mid e]$, used for the frustum corners and for mouse picking.

Check. Default camera e = (0, 0, 2.8), looking at the origin with u = (0, 1, 0): f = (0, 0, −1), s = f × u = (1, 0, 0), v = s × f = (0, 1, 0), b = (0, 0, 1), so V is a translation by −2.8 in z. For the orbit camera of the screenshots (yaw 25°, pitch 12°, eye (1.16, 0.58, 2.48)) V·eye = (0, 0, 0) and V·target = (0, 0, −2.8). **Degenerate case:** if u is parallel to f then f × u = 0 and cannot be normalised; the code tests $|f \times u| < 10^{-6}$ and substitutes an axis not parallel to f.

Projection (WebGPU depth z in [0, 1])

Perspective, vertical field of view φ, aspect a, near n, far f: x and y are scaled by 1/(a·t) and 1/t with t = tan(φ/2), and w = −z. The depth row is z_clip = A·z + B, z_ndc = (A·z + B)/(−z). Requiring z_ndc = 0 at z = −n and 1 at z = −f gives B = A·n and −A·f + A·n = f, hence **A = f/(n − f), B = n·f/(n − f)**. Orthographic: x, y scaled by 2/(r − l), 2/(t − b) with offsets, and depth z_ndc = A·z + B with A = 1/(n − f), B = n/(n − f), w = 1. The inspection camera draws the main camera's eight frustum corners (plane at distance d is z = −d,

half-height d·tan(φ/2) or orthoHeight/2, half-width = half-height·a), moved to world space with V^{-1}.

```
    [ 1/(a·t)    0        0         0 ]
P = [       0  1/t        0         0 ]
    [       0    0  f/(n−f)  nf/(n−f) ]
    [       0    0       −1         0 ]
```

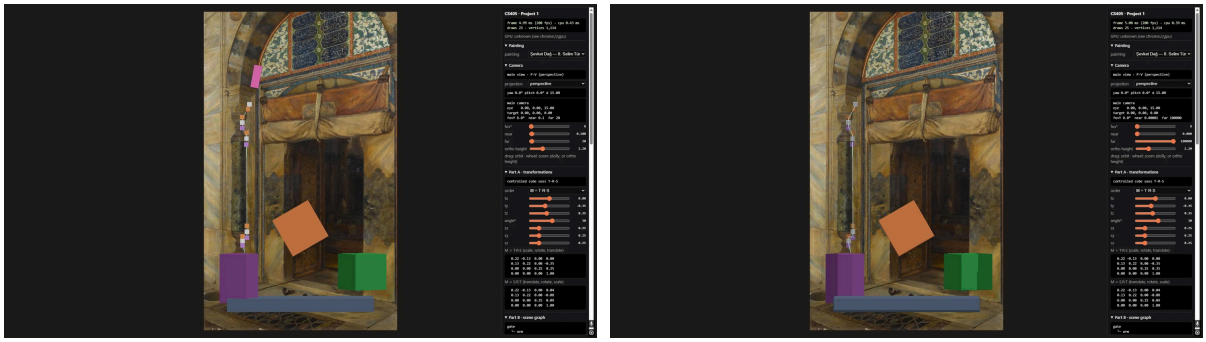


Same scene and camera. Left: perspective. Middle: orthographic (parallel edges stay parallel). Right: the main camera's frustum seen from the fixed inspection camera (near plane yellow, far plane orange).

Near/far values and depth precision

Depth is stored as z_ndc(d) = f(d − n)/(d(f − n)) with d = −z, whose slope is f·n/((f − n)d^2). A 24-bit buffer (step δ = 2^{-24} ≈ $6{\cdot}10^{-8}$) therefore separates two surfaces only if their distances differ by about **Δd ≈ δ(f − n)d^2/(f·n) ≈ d^2/(n·2^{24})**: it grows with d^2 and shrinks with n, while f hardly matters. Experiment: camera 15 units away with a field of view of 8° (so the painting fills the screen) and the curve lines only 0.0002 in front of the painting.

setting	near	far	distance	Δd (24-bit estimate)	gap to curve
screenshot "good"	0.1	20	15	**1.3e−4**	0.0002
screenshot "bad"	0.00001	100000	15	**1.3**	0.0002
application default	0.1	100	2.8	4.7e−6	0.012

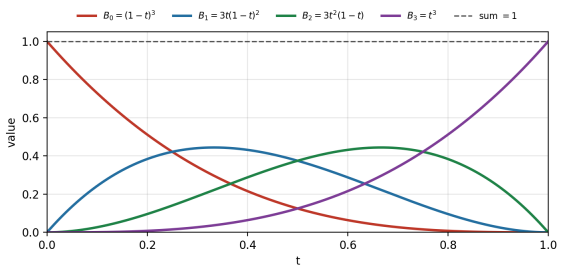


Same view, only near/far change. Left: near 0.1, far 20, clean. Right: near 0.00001, far 100000, depth-buffer errors (jagged cube edges, broken curve lines).

With near 0.1 / far 20 the resolution (≈ 0.00013) is finer than the gap and the picture is clean; with near 0.00001 / far 100000 it is ≈ 1.3, thousands of times coarser than the gap, and surfaces fight. **Chosen values: near 0.1, far 100.** The nearest visible object is more than 2 units away, and the resolution at the painting (≈ $5{\cdot}10^{-6}$) is far below the 0.012 gap between painting and curve lines. This is a worst-case estimate for a 24-bit buffer (the GPU picks the exact format); the observed failure agrees with it.

Part D: Bézier curves and the traced shape

Basis functions. A cubic Bézier curve is P(t) = Σ $B_i(t)\cdot P_i$ with $B_i(t)$ = C(3, i)$(1-t)^{3-i}t^i$: **$B_0 = (1-t)^3$, $B_1 = 3t(1-t)^2$, $B_2 = 3t^2(1-t)$, $B_3 = t^3$.** They are non-negative on [0, 1] and sum to 1 because $(1 - t + t)^3 = 1$ (binomial theorem); at t = ½ the weights are 1/8, 3/8, 3/8, 1/8.



The four cubic Bernstein basis functions and their sum (dashed, always 1).

Start and end points. At t = 0: $B_0(0) = 1$, while $B_1(0) = 3{\cdot}0{\cdot}1^2 = 0$, $B_2(0) = 3{\cdot}0^2{\cdot}1 = 0$ and $B_3(0) = 0^3 = 0$ (each contains a factor t), so $P(0) = P_0$. At t = 1: $B_3(1) = 1$, while $B_0(1) = 0^3 = 0$, $B_1(1) = 3{\cdot}1{\cdot}0^2 = 0$ and $B_2(1) = 3{\cdot}1^2{\cdot}0 = 0$ (each contains a factor 1 − t), so **$P(1) = P_3$**. The curve interpolates its end points; P_1 and P_2 only pull the shape.

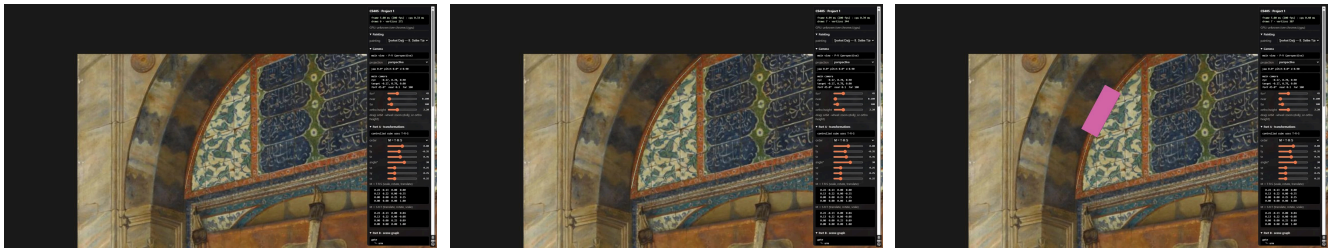
C^1 condition. $P'(t) = 3[(1-t)^2(P_1 - P_0) + 2(1-t)t(P_2 - P_1) + t^2(P_3 - P_2)]$, so $P'(0) = 3(P_1 - P_0)$ and $P'(1) = 3(P_3 - P_2)$. Two segments a and b joined at t = 1 of a and t = 0 of b are C^0 if $P_3{}^a = P_0{}^b$ and C^1 if also $3(P_3{}^a - P_2{}^a) = 3(P_1{}^b - P_0{}^b)$, i.e. **$P_1{}^b = 2{\cdot}P_3{}^a - P_2{}^a$**: $P_2{}^a$, the joint

and P_1^b are collinear with the joint in the middle. The application enforces this by construction (every later segment takes P_0 from the previous P_3 and P_1 from the mirror $2 \cdot P_3 - P_2$), so only P_2 and P_3 are free and dragging a joint carries its neighbours.

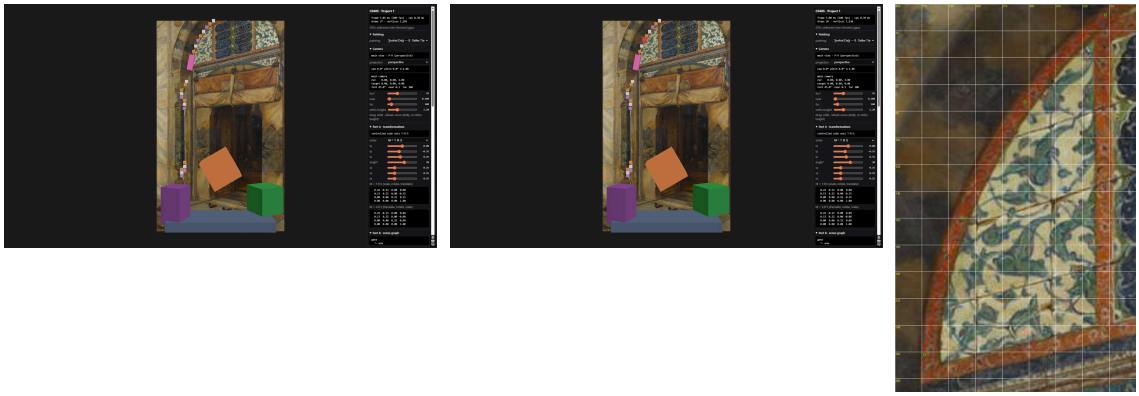
joint of the traced curve (u, v)	$P_3^a = P_0^b$	$2 \cdot P_3^a - P_2^a$	P_1^b in curve.json
joint 1	(0.30848, 0.125)	(0.32519, 0.09922)	(0.32519, 0.09922)
joint 2	(0.36889, 0.05703)	(0.39332, 0.03672)	(0.39332, 0.03672)

de Casteljau evaluates the same curve by repeated interpolation, $Q_i^r = (1 - t)Q_i^{r-1} + t \cdot Q_{i+1}^{r-1}$. For segment 2 at t = 0.4 the three rounds give (0.31517, 0.11469), (0.3329, 0.09047), (0.35424, 0.06922), then (0.32226, 0.105), (0.34144, 0.08197), then the point (0.32993, 0.09579), equal to $\Sigma\, B_i(0.4) \cdot P_i$. **Catmull–Rom** (second curve family) $p(t) = \frac{1}{2}[2p_1 + (p_2 - p_0)t + (2p_0 - 5p_1 + 4p_2 - p_3)t^2 + (3p_1 - p_0 - 3p_2 + p_3)t^3]$ passes through p_1 and p_2 with tangents $(p_2 - p_0)/2$ and $(p_3 - p_1)/2$; drawn through the four joints it stays close to the Bézier curve because the traced edge is smooth (a uniform B-spline basis is also implemented).

Traced shape and interaction. The curve (3 cubic segments, C^1 at both joints) traces the orange border of the tiled arch. Control points are placed and dragged with the mouse in the plane of the painting: a ray from the camera through the mouse position is intersected with z = 0. A pink box moves along the curve and is rotated by atan2(tangent.y, tangent.x) so that it points along the tangent. The curve lines sit 0.012 in front of the painting to avoid z-fighting. The exported JSON is in Appendix A.



Left: traced shape over the painting. Middle: Catmull–Rom overlay next to the Bézier curve. Right: animated box on the curve, oriented by the tangent.



Left: control points (white joints, orange free points, purple mirrored points that make the joints C^1) and control polygons. Middle: de Casteljau construction of segment 2 at t = 0.4. Right: accuracy check, the curve (red) on the full-resolution painting with a 20-pixel grid; it stays inside the orange band.

Bonus: surface of revolution

The right edge of the painted marble column is traced as a second curve (5 Bézier segments, C^1 at 4 joints). It is sampled into points (r, y), r = distance from the column's centre line, ordered top to bottom, and each point is turned about the vertical axis by $\theta_j = 2\pi j/N$: position $(r \cdot \cos\theta_j,\ y,\ r \cdot \sin\theta_j)$. **Normals:** the profile tangent (dr, dy) is estimated by finite differences (central inside, one-sided at the ends); the outward normal in the (r, y) plane is $(-dy, dr)/|(-dy, dr)|$ (a cylinder, tangent (0, −1), gives (1, 0)), turned by θ_j to $(n_r \cdot \cos\theta_j,\ n_y,\ n_r \cdot \sin\theta_j)$. Each profile segment and angle gives a quad, split into two triangles, so m profile points give $(m - 1) \cdot N \cdot 6$ vertices ($120 \cdot 48 \cdot 6 = 34{,}560$). The mesh is rebuilt live when the profile curve or N changes.



The 3D column (grey) pulled out of the painting (z = 0.5), in front of the painted column.

Measurements

GPU: NVIDIA GeForce RTX 3070 Ti Laptop GPU (discrete), driver 32.0.16.1074, WebGPU on the D3D12 backend, Brave (Chromium 157) on Windows 11, hardware accelerated. The page ran on a 200 Hz display, so 5.00 ms is the display refresh and not a GPU limit;

CPU time per frame is the informative number.

setup	frame	CPU per frame	draw calls	vertices per frame
everything on	4.99 ms (200 fps)	0.52 ms	44	36,274
3D column off	5.00 ms (200 fps)	0.51 ms	43	1,714
curves, column and lamp off	5.00 ms (200 fps)	0.43 ms	5	150

Vertex check: 36,274 − 1,714 = 34,560 = the revolved column (120 × 48 × 6); 150 = the painting (6) + four cubes (4 × 36).

What went wrong and how I found it

- **The first curve joint was hidden.** The gold lamp bracket sits exactly over the lower end of the traced arch, so joint 1 was missing from my first screenshots. I noticed when comparing the screenshot with the control-point list and added a "show lamp" switch.
- **My first trace followed a different edge** (the outer stone edge). I overlaid the curve on the full-resolution painting with a pixel grid, saw that the orange tile border is a much clearer edge, and moved the trace onto it.
- **The depth experiment showed nothing at first.** With the normal 0.012 gap, even near = 0.001 and far = 1000 at distance 15 produced no visible artefact. The estimate $\Delta d \approx d^2/(n \cdot 2^{24})$ shows a failure appears only when Δd exceeds the gap, so I added a setting for the gap (0.0002) and pushed near to 0.00001, which made the failure obvious.
- **The GPU name read "unknown" in the panel** because the browser hides adapter details; I took it from the browser's GPU information page. **Frame time looked constant at 5 ms** because it follows the 200 Hz display, so I also report CPU time.

Appendix A: exported curve.json (normalised painting coordinates, origin top-left, v down)


```
{
  "painting": "selim-turbesi",
  "kind": "bezier-c1",
  "segments": [
    [
      [0.27249357326478146, 0.20468750000000002],
      [0.2802056555269923, 0.1796875],
      [0.2917737789203085, 0.15078124999999998],
      [0.30848329048843187, 0.125]
    ],
    [
      [0.30848329048843187, 0.125],
      [0.32519280205655526, 0.09921875000000002],
      [0.3444730077120823, 0.07734374999999999],
      [0.36889460154241643, 0.05703124999999998]
    ],
    [
      [0.36889460154241643, 0.05703124999999998],
      [0.3933161953727506, 0.03671874999999997],
      [0.41773778920308485, 0.021093750000000022],
      [0.4460154241645245, 0.006249999999999978]
    ]
  ]
}
```


Appendix B: AI use

tool	what I asked	what I kept
Claude (Claude Code, Claude Sonnet 5.5)	Explanations of the maths (matrix conventions, lookAt, projection, Bézier curves, C^1 continuity, surface of revolution) and debugging help.	Used for understanding. I wrote the graded maths myself: vec3.js, mat4.js, node.js (updateWorld), compose.js, the five local matrices in animate(), cameraMath.js (orbitEye, frustumCorners, screenToPaintingPoint), curves.js, curveToJSON and revolve.js.
Claude (Claude Code)	Rendering and application plumbing: mesh and line pipelines with their WGSL shaders, the control panel, the lamp tree set-up, camera orbit input and frustum drawing, the curve editor, the surface-of-revolution view, address-bar presets, the README.	Kept and integrated; these call my maths functions.
Claude (Claude Code)	Unit tests for my functions, a starting guess for the curve points (checked against the painting), and the drafting and layout of this report.	Kept after checking. Numbers in the report were computed by a script and compared with the matrices shown by the running application.